



دانشگاه تهران
دانشکده برق و کامپیوتر
آزمایشگاه معماری کامپیوتر
گزارش کار (پردازنده ARM)

اعضای گروه:

امیرحسین عارف زاده - 810101604

امین آقاکثیری - 810101381

مهدی قزلباش - 810100269

* تمامی کد ها از لینک های قرار داده شده در متن گزارش در گیت هاب قابل دسترسی هستند.

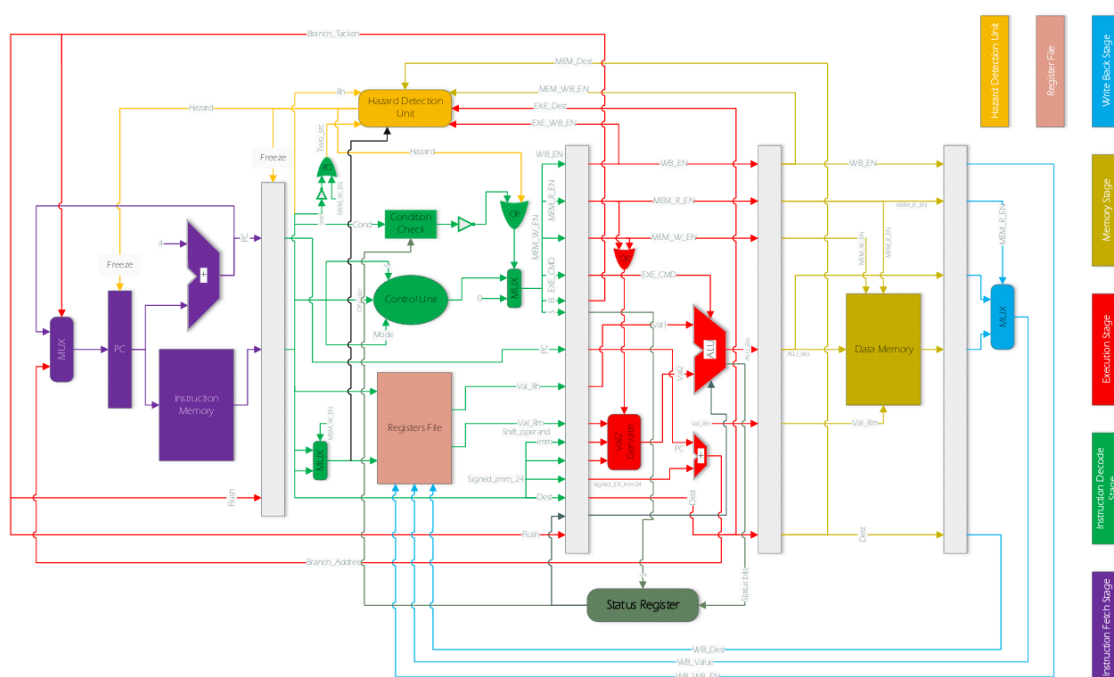
مقدمه

در این درس قصد داریم با پیاده سازی یک پردازنده ARM، درک عمیق تریاز سازوکار پردازنده هاپیدا کنیم. برای این منظور، پروژه به چند بخش تقسیم شده است که هر کدام از این بخش هادر نهایت برای ساخت محصول نهایی به هم متصل می شوند. پردازنده مورد نظر از پنج مرحله اصلی تشکیل شده است :

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execution (EXE)
- Memory
- Write Back (WB)

بین هر یک از این مراحل، یک رجیستر موقت قرار داده شده است تا اطلاعات هر مرحله را در چرخه کلاک بعدی به مرحله بعدی منتقل کند .

در پنج بخش ابتدایی گزارش، نحوه پیاده سازی هر یک از این مراحل توضیح داده شده است. در ادامه، تغییراتی که به منظور بهبود عملکرد پردازنده اعمال شده اند، مانند افزودن مکانیزم Forwarding به همراه جزئیات آنها شرح داده می شود. ساختار کلی پردازنده به صورت زیر است :



شکل پردازنده ARM

ب. حافظه دستورالعمل

حافظه دستورالعمل (Instruction Memory) شامل دستورالعمل‌هایی است که باید اجرا شوند. در این پیاده‌سازی، حافظه دستورالعمل به صورت یک ماژول آماده در Vivado است که می‌تواند دستورالعمل‌ها را از یک فایل بخواند.

ج. جمع‌کننده

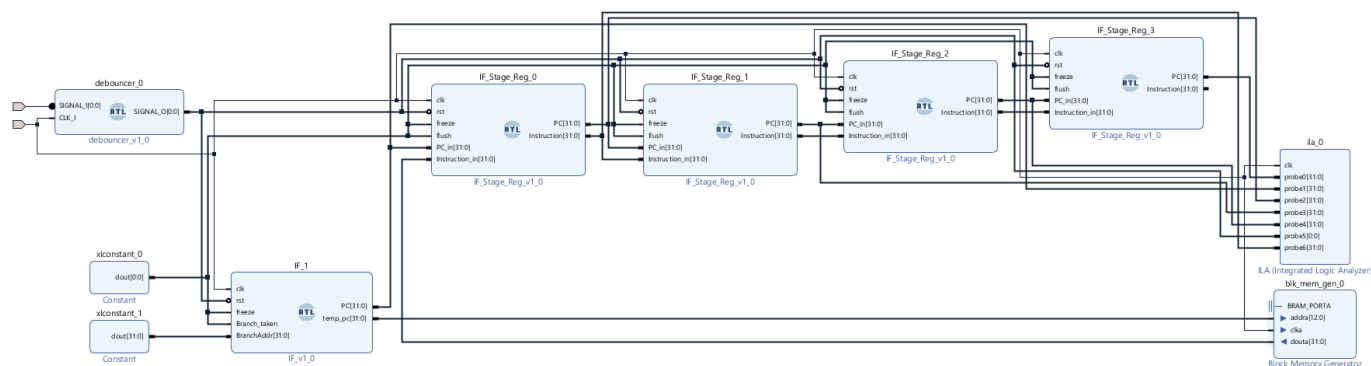
جمع‌کننده در این مرحله برای افزایش مقدار PC به کار می‌رود. پس از هر بار واکنشی دستورالعمل، مقدار PC توسط جمع‌کننده افزایش می‌یابد تا به آدرس دستورالعمل بعدی اشاره کند. این افزایش معمولاً به اندازه ۴ بایت (یک دستورالعمل) است، زیرا هر دستورالعمل ۳۲ بیتی (۴ بایت) است.

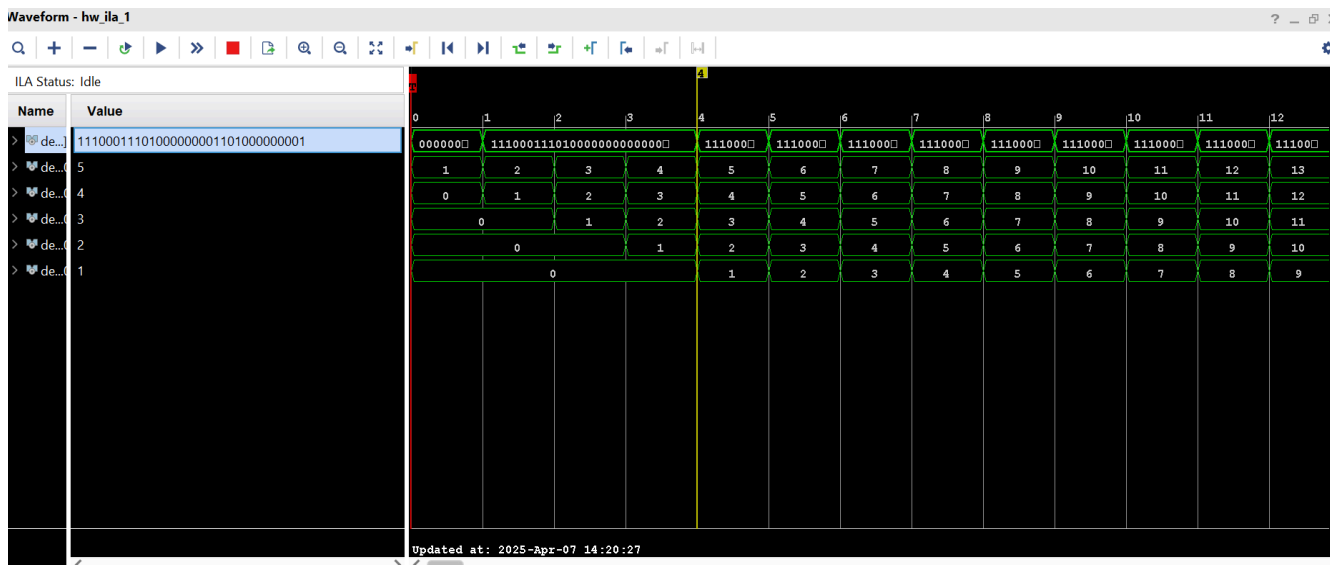
د. مالتی پلکسر (Mux)

مالتی پلکسر برای انتخاب بین حالت عادی و یا استفاده از دستورالعمل branch (branch) استفاده می‌شود که بر اساس سیگنال کنترل‌کننده branch_taken تصمیم می‌گیرد که کدام یک از این دو مقدار را به عنوان خروجی انتخاب کند.

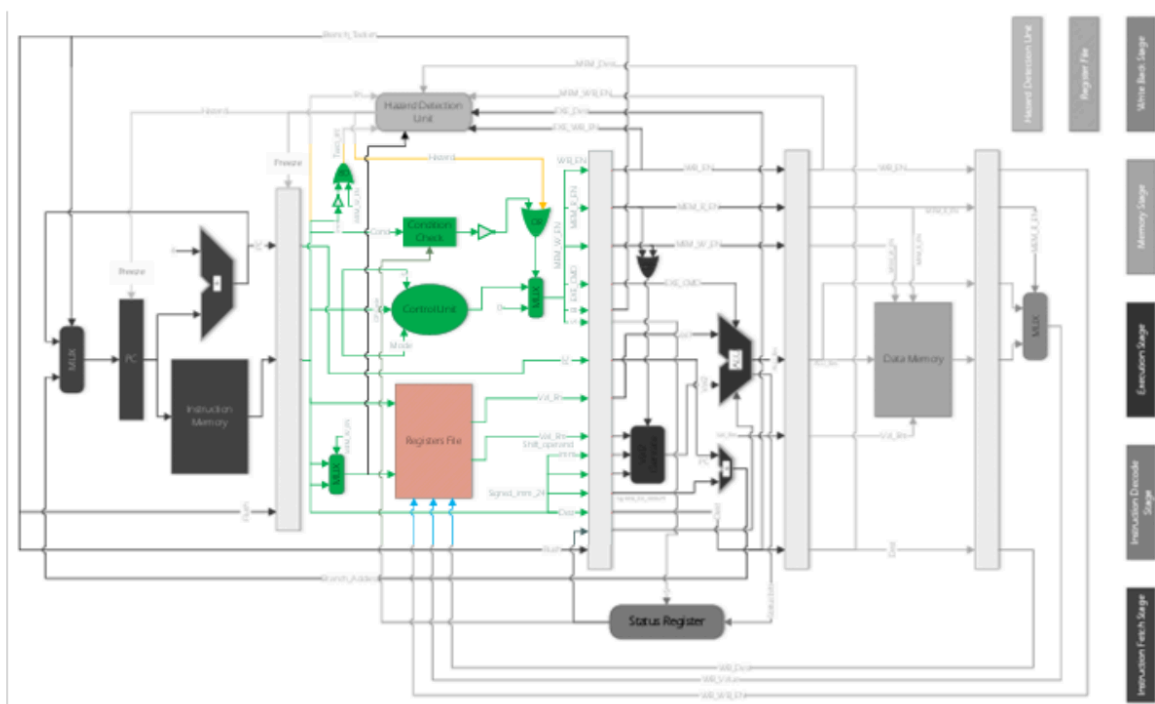
ه. نتایج

در این مرحله، دستورالعمل‌ها به درستی از حافظه واکنشی می‌شوند و به همراه مقدار PC به مرحله بعدی ارسال می‌شوند. برای اطمینان از صحت پیاده‌سازی، رجیسترهای میانی بین مراحل مختلف خط لوله ایجاد شده‌اند. این رجیسترها سیگنال‌های PC و instruction را نگه می‌دارند و انتقال آن‌ها را بین مراحل خط لوله مدیریت می‌کنند. این کار به ما امکان می‌دهد تا حرکت پله پله دستورالعمل‌ها و PC را در مراحل مختلف خط لوله مشاهده و تایید کنیم که به صورت زیر هستند:





مرحله دوم : پیاده سازی ID Stage



مرحله ی ID جایی است که دستورالعمل واکشی شده decode می شود. این مرحله شامل واحد کنترل است که دستورالعمل را رمزگشایی می کند. یک نمای کلی از آنچه در این مرحله اتفاق می افتد به شرح زیر است:

الف. شناسایی و رمزگشایی دستورالعمل

دستورالعمل واکنشی شده شناسایی می‌شود. این شامل تعیین نوع دستورالعمل و عملیات‌هایی است که انجام می‌دهد. این کار با تقسیم دستورالعمل به بخش‌های مختلف بر اساس فایل ارائه شده به ما انجام می‌شود سپس دستورالعمل رمزگشایی می‌شود. دستورالعمل‌ها معمولاً به شکلی ذخیره می‌شوند که نیاز به ترجمه (رمزگشایی) دارند تا به سیگنال‌های کنترل برای اجرای دستورالعمل تبدیل شوند. این رمزگشایی یک لایه انتزاعی بین سخت‌افزار و نرم‌افزار ایجاد می‌کند که امکان تغییر سیگنال‌های کنترل مورد استفاده برای اجرا را بدون شکستن سازگاری باینری فراهم می‌کند.

ب. واحد کنترل

ماژول واحد کنترل (control unit) در ترکیب با ماژول بررسی شرط (Condition Check Module)، به ما این امکان را می‌دهد تا یک پردازنده شرطی داشته باشیم. ماژول بررسی شرط بر اساس جدول زیر کار می‌کند:

Opcode [31:28]	Mnemonic extension	Meaning	Condition flag state
0000	EQ	Equal	Z set
0001	NE	Not equal	Z clear
0010	CS/HS	Carry set/unsigned higher or same	C set
0011	CC/LO	Carry clear/unsigned lower	C clear
0100	MI	Minus/negative	N set
0101	PL	Plus/positive or zero	N clear
0110	VS	Overflow	V set
0111	VC	No overflow	V clear
1000	HI	Unsigned higher	C set and Z clear
1001	LS	Unsigned lower or same	C clear or Z set
1010	GE	Signed greater than or equal	N set and V set, or N clear and V clear (N == V)
1011	LT	Signed less than	N set and V clear, or N clear and V set (N != V)
1100	GT	Signed greater than	Z clear, and either N set and V set, or N clear and V clear (Z == 0, N == V)
1101	LE	Signed less than or equal	Z set, or N set and V clear, or N clear and V set (Z == 1 or N != V)
1110	AL	Always (unconditional)	-
1111	-	See Condition code 0b1111	-

همچنین ماژول واحد کنترل از قوانین جدول زیر جهت تعیین دستور اجرایی برای ماژول ALU و همچنین سایر سیگنال‌های مورد نیاز مانند B، MEM_W_EN، MEM_R_EN، WB_EN و S پیروی می‌کند.

R-type Instructions		Description	Bits							
			31:28	27:26	25	24:21	20	19:16	15:12	11:00
			Cond.	Mode	I	OP-Code	S	Rn	Rd	shifter operand
0	NOP ^{۱۱}	No Operation	1110	00	0	0000	0	0000	0000	000000000000
1	MOV	Move	cond	00	I	1101	S	0000	Rd	shifter operand
2	MVN ^{۱۲}	Move NOT	cond	00	I	1111	S	0000	Rd	shifter operand
3	ADD	Add	cond	00	I	0100	S	Rn	Rd	shifter operand
4	ADC	Add with Carry	cond	00	I	0101	S	Rn	Rd	shifter operand
5	SUB	Subtraction	cond	00	I	0010	S	Rn	Rd	shifter operand
6	SBC	Subtract with Carry	cond	00	I	0110	S	Rn	Rd	shifter operand
7	AND	And	cond	00	I	0000	S	Rn	Rd	shifter operand
8	ORR	Or	cond	00	I	1100	S	Rn	Rd	shifter operand
9	EOR	Exclusive OR	cond	00	I	0001	S	Rn	Rd	shifter operand
10	CMP	Compare	cond	00	I	1010	1	Rn	0000	shifter operand
11	TST ^{۱۳}	Test	cond	00	I	1000	1	Rn	0000	shifter operand
12	LDR	Load Register	cond	01	0	0100	1	Rn	Rd	offset_12
13	STR	Store Register	cond	01	0	0100	0	Rn	Rd	offset_12
14	B	Branch	cond	10	1	0	signed_immed_24			

همه این سیگنال‌ها بر اساس نوع عملیاتی که در حال اجرا است فعال یا غیرفعال می‌شوند. اگر شرایط (خروجی ماژول بررسی شرط) برقرار نباشد یا با hazard مواجه شویم، همه این سیگنال‌ها به ۰ تغییر می‌کنند که نشان‌دهنده NOP (عدم انجام عملیات) است.

ج. واکنشی عملوندها

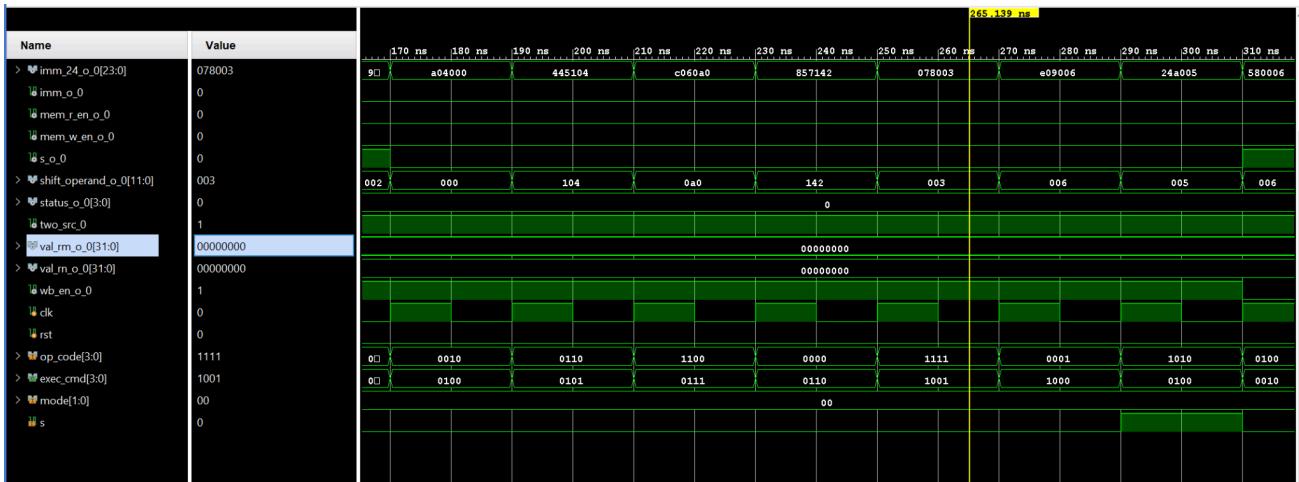
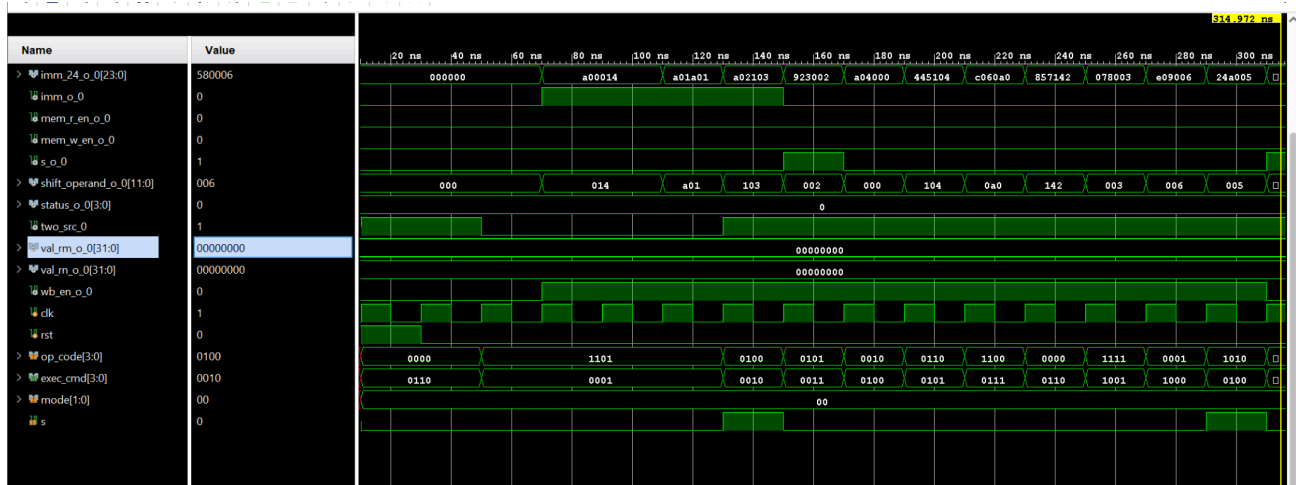
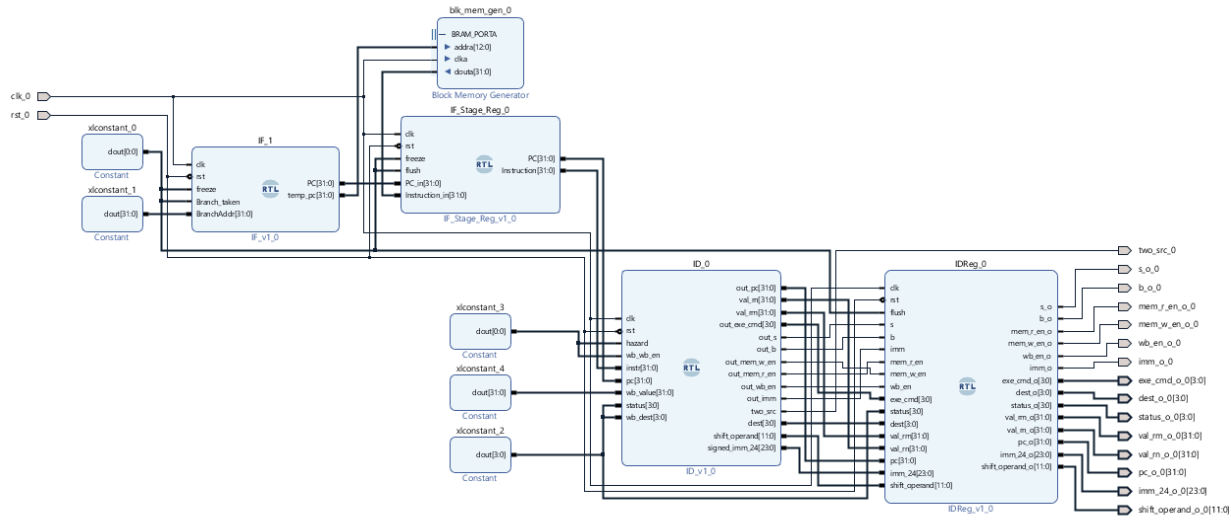
در این مرحله، عملوندهای دستورالعمل واکنشی می‌شوند. این فرآیند شامل خواندن مقادیر از رجیسترها یا حافظه است. [ماژول Register File](#) به صورت همگام با کلاک نوشته می‌شود، اما خواندن به صورت ناهمگام انجام می‌گیرد. ورودی‌ها بر اساس decode دستورالعمل که قبلاً انجام شده، تعیین می‌شوند. با این حال، mux برای ورودی دوم این ماژول برای دستوراتی مانند STR که از src2 استفاده می‌کنند، طراحی شده است. سایر دستورات از Rm برای ورودی دوم استفاده می‌کنند.

د. تشخیص هازارد

مرحله decode در خط لوله همچنین می‌تواند انواع هازاردهای داده‌ای و ساختاری را نسبت به دستورالعمل‌های قبلی تشخیص دهد. در صورت نیاز، اقداماتی مانند استفاده از result forwarding یا اقدامات بازدارنده مانند

متوقف کردن خط لوله یا stall کردن انجام می‌شود. این بخش به طور کامل در آینده توضیح داده خواهد شد.

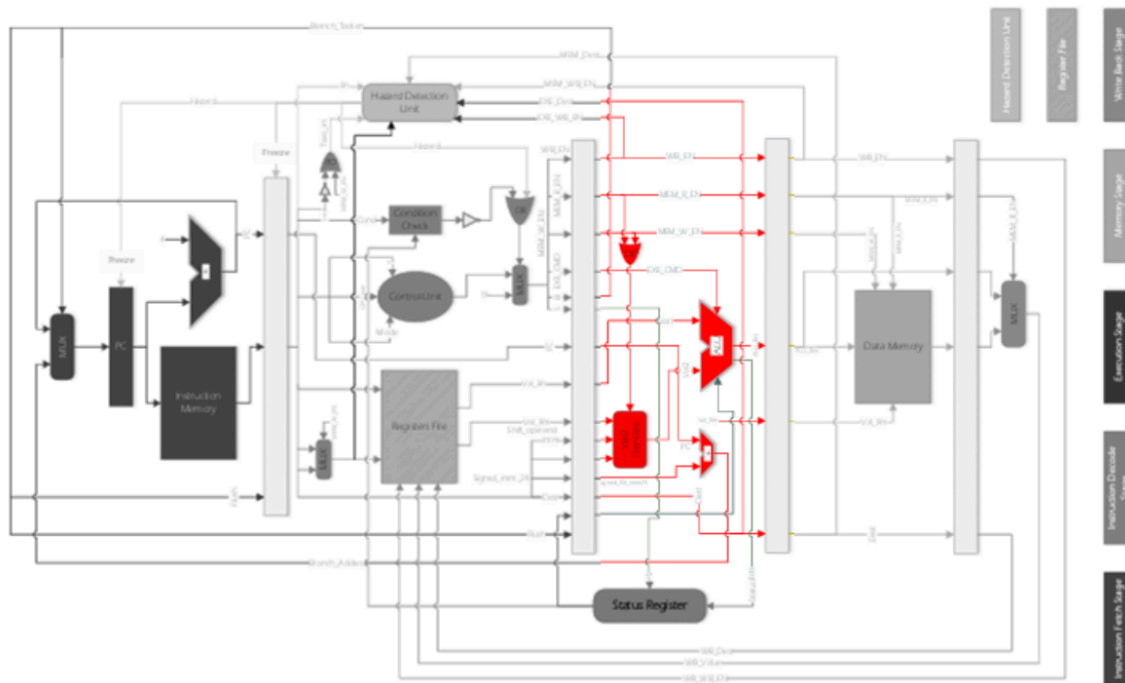
۵. نتایج



مرحله سوم : پیاده سازی EXE و WB

مرحله‌ی Execution (EXE) و Write Back (WB) دو مرحله مهم در خط لوله پردازنده هستند که به ترتیب عملیات محاسباتی و ذخیره نتایج را انجام می‌دهند. در ادامه، این مراحل به صورت کامل توضیح داده می‌شوند.

مرحله EXE



مرحله‌ی EXE جایی است که عملیات محاسباتی و منطقی بر روی داده‌ها انجام می‌شود. این مرحله شامل واحدهای اصلی زیر است:

الف. ALU

ماژول ALU عملیات محاسباتی و منطقی را بر اساس سیگنال‌های EXE_CMD انجام می‌دهد. دو ورودی اصلی دارد: مقدار رجیستر و Rn که توسط ماژول val2generate تولید می‌شود. این ورودی می‌تواند یک مقدار ثابت (immediate) یا نتیجه یک عملیات شیفت باشد.

این ماژول علاوه بر نتیجه عملیات، بیت‌های وضعیت (Status Flags) مانند C(Carry) و Z(Zero) و V(Overflow) و N(Negative) را نیز تولید می‌کند. این بیت‌ها برای دستورات شرطی و تصمیم‌گیری‌های بعدی استفاده می‌شوند.

ب. Val2Generate

[ماژول Val2Generate](#) وظیفه محاسبه اپرند دوم ALU را بر عهده دارد. این ماژول دارای چهار ورودی اصلی است: MemInst که از OR شدن EN_R_MEM و EN_W_MEM به دست می‌آید و نشان‌دهنده دستورات مرتبط با حافظه است، Imm که بیت ۲۵ دستور است و مشخص‌کننده نوع اپرند دوم است، ShifterOperand که ۱۲ بیت سمت راست دستور است و ValRm که مقدار رجیستر دوم است. خروجی این ماژول یک مقدار ۳۲ بیتی است که به عنوان ورودی دوم ALU استفاده می‌شود.

مقدار	توضیحات	وضعیت شیفت
00	Logical shift left	LSL
01	Logical shift right	LSR
10	Arithmetic shift right	ASR
11	Rotate right	ROR

این ماژول در چند حالت مختلف عمل می‌کند. اگر دستور مربوط به حافظه باشد، خروجی مقدار Extend-Sign شده ۱۲ بیت ShifterOperand است. اگر Imm برابر با ۱ باشد، ۸ بیت سمت راست ShifterOperand به صورت دایره‌ای شیفت می‌خورد. در حالت دیگر، اگر Imm برابر با ۰ باشد، ShifterOperand نوع شیفت مانند LSR، LSL، ROR و مقدار شیفت را مشخص می‌کند و ValRm بر اساس آن شیفت داده می‌شود. این ماژول به‌طور موثر ورودی دوم ALU را بر اساس نوع دستور و پارامترهای آن محاسبه می‌کند.

ج. ثبات وضعیت

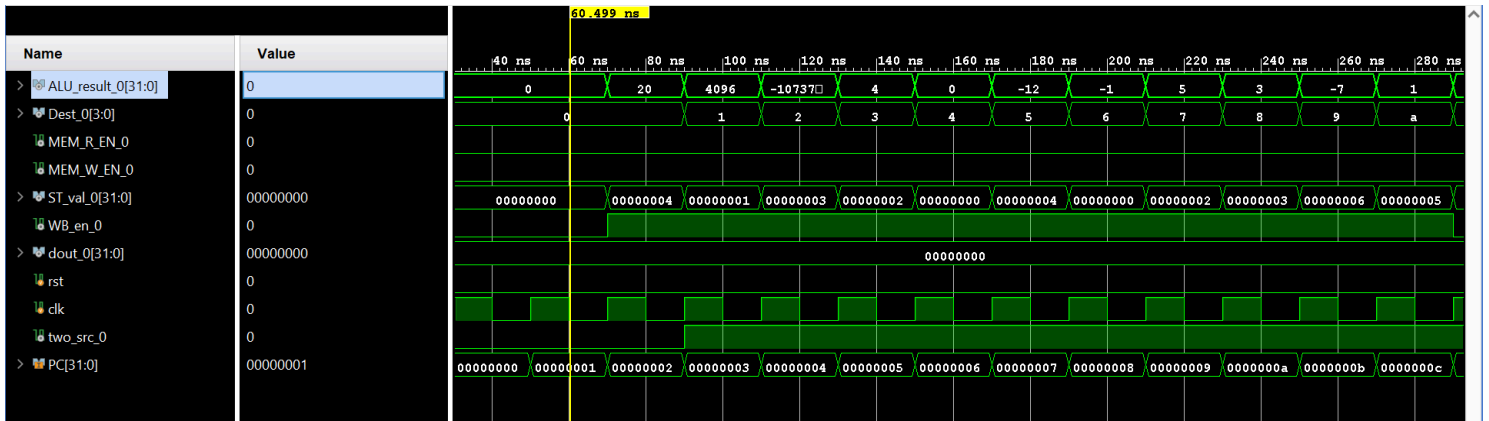
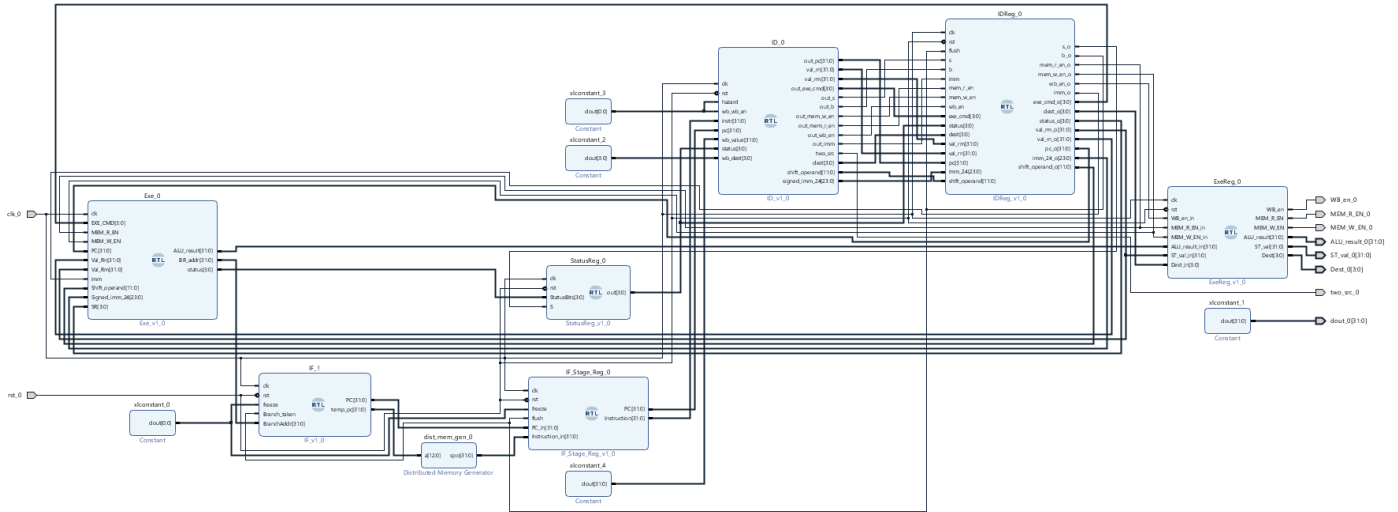
[ثبات وضعیت](#) برای نگهداری بیت‌های وضعیت (C, V, N, Z) استفاده می‌شود. این بیت‌ها از خروجی ALU دریافت می‌شوند و برای دستورات شرطی و تصمیم‌گیری‌های بعدی مورد استفاده قرار می‌گیرند.

د. پیدا کردن آدرس پرش

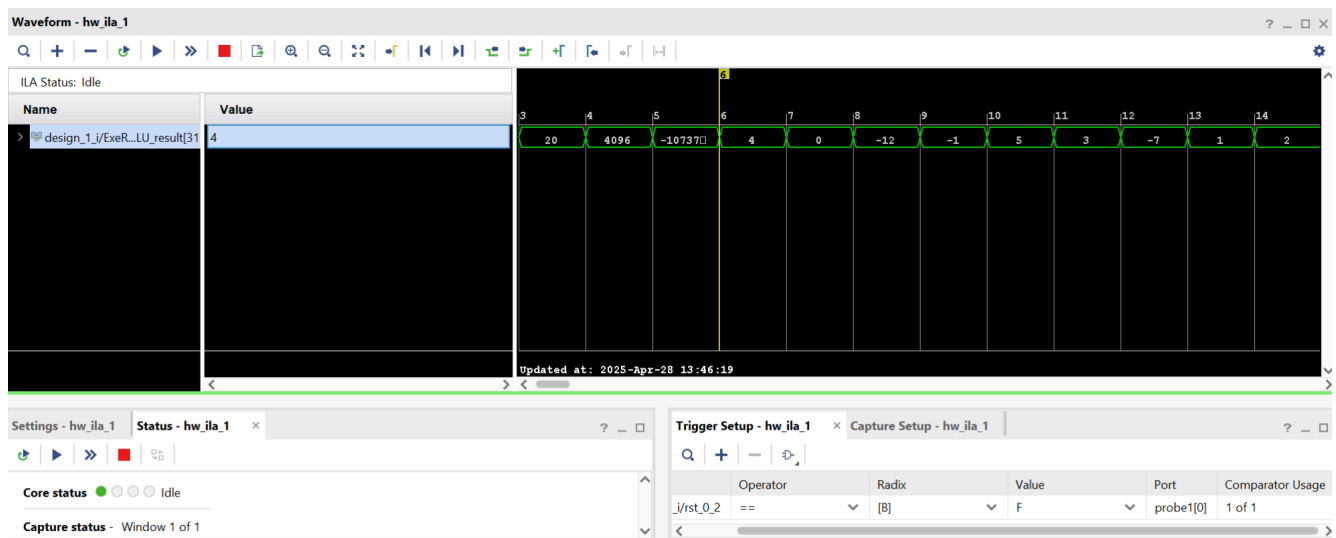
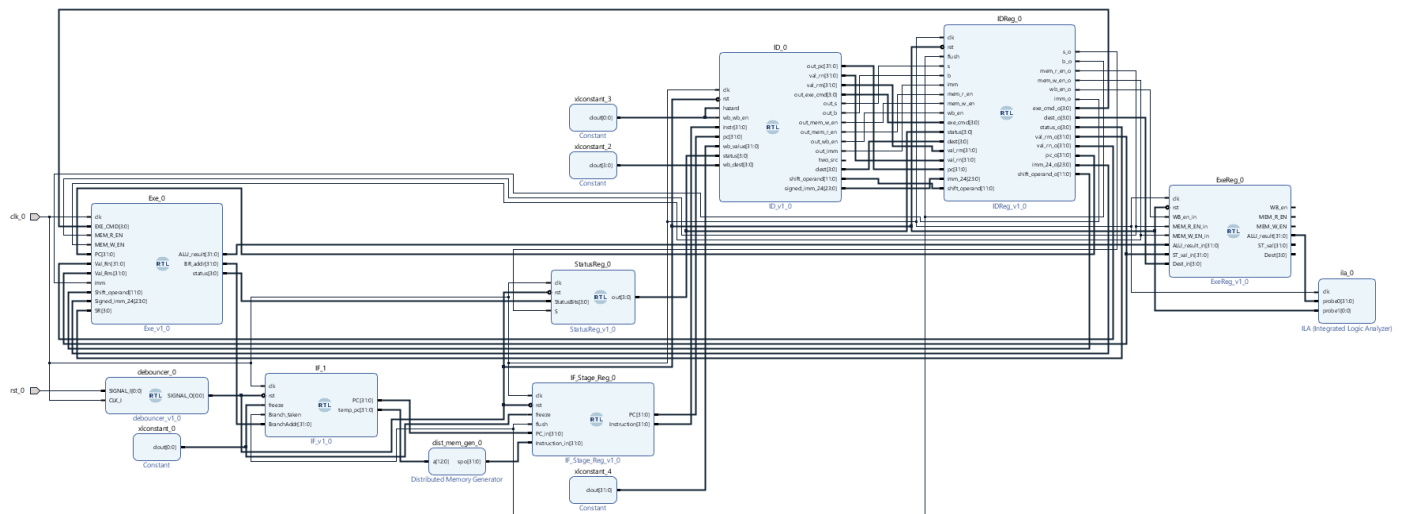
[ماژول Adder](#) یک جمع‌کننده ۳۲ بیتی است که برای محاسبه آدرس Branch مورد استفاده قرار می‌گیرد. این ماژول دو ورودی اصلی دارد: مقدار $PC + 4$ که آدرس دستورات عمل بعدی در حالت عادی است و مقدار Extend Sign شده ۲۴ بیت سمت راست دستور Branch که نشان‌دهنده آفست پرش است. خروجی این ماژول حاصل جمع این دو مقدار است که به مرحله اول پایپلاین یعنی IF باز می‌گردد. این آدرس محاسبه شده به عنوان آدرس هدف پرش استفاده می‌شود و به PC منتقل می‌شود تا دستورات عمل بعدی از آدرس جدید واکنشی شود. این ماژول نقش کلیدی در اجرای دستورات پرش و تغییر مسیر اجرای برنامه دارد.

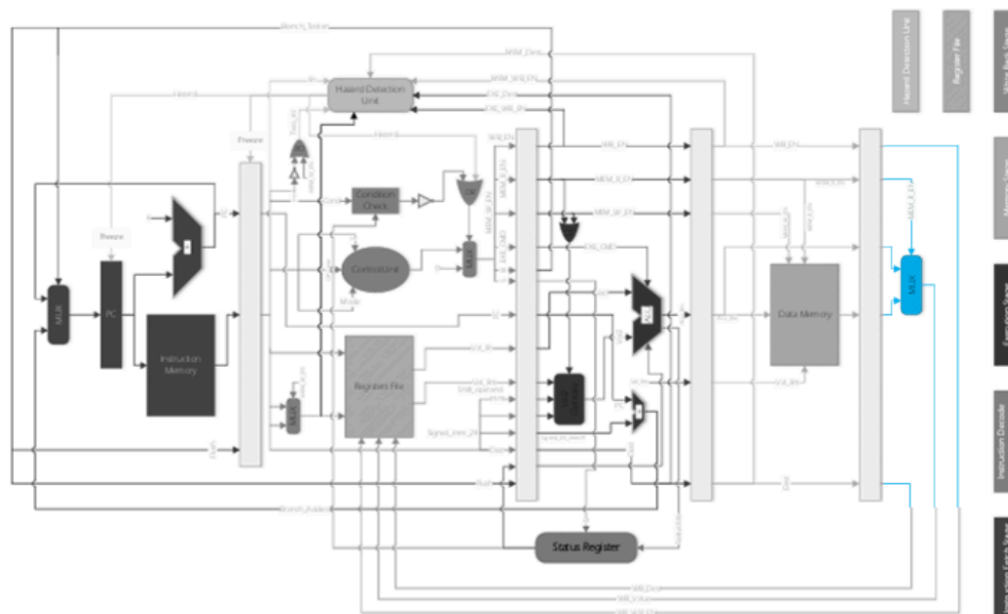
٥. نتائج

:Simulation



:On board



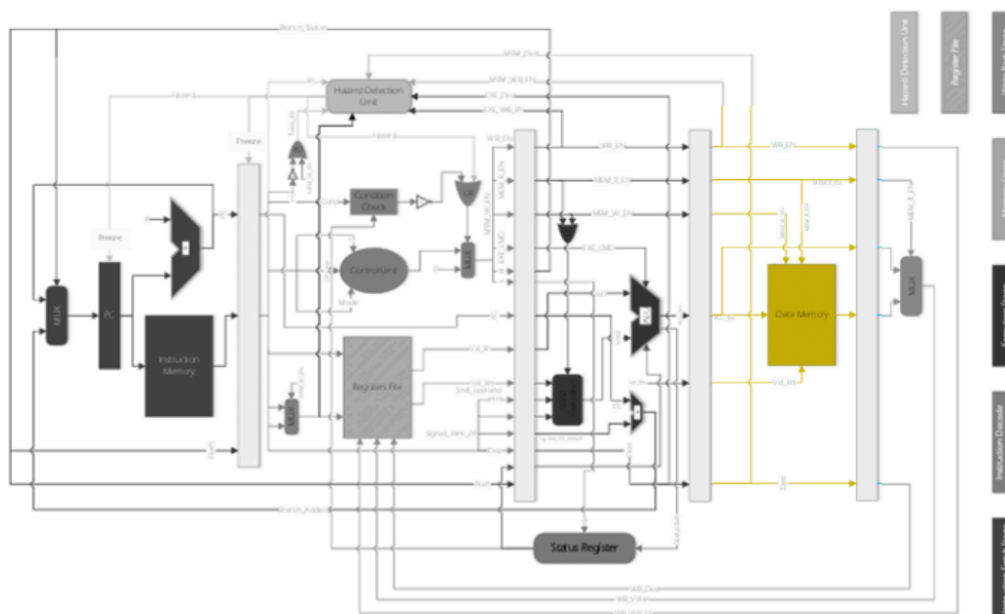


مرحله WB یا Write Back آخرین مرحله در خط لوله پردازنده است که در آن نتایج عملیات به رجیسترهای عمومی بازنویسی می‌شوند. این مرحله شامل واحدهای اصلی زیر است:

در این مرحله، یک مالتی‌پلکسر (MUX) وجود دارد که تصمیم می‌گیرد داده‌های خروجی از کدام منبع به رجیسترهای عمومی ارسال شوند. اگر سیگنال MEM_R_EN فعال باشد، داده از حافظه (Memory) خوانده می‌شود و به رجیسترها ارسال می‌شود. اگر MEM_R_EN غیرفعال باشد، داده از مرحله EXE یعنی خروجی ALU به رجیسترها ارسال می‌شود. این انتخاب بر اساس سیگنال‌های کنترل‌کننده‌ای مانند MEM_R_EN و WB_EN انجام می‌شود.

نتایج عملیات، اعم از داده‌های محاسبه‌شده توسط ALU یا داده‌های خوانده‌شده از حافظه، در رجیسترهای عمومی بازنویسی می‌شوند. این کار بر اساس آدرس مقصد (Destination Register) که در مرحله ID تعیین شده است، انجام می‌شود. این مرحله اطمینان حاصل می‌کند که نتایج نهایی عملیات به درستی در رجیسترهای مورد نظر ذخیره می‌شوند و برای دستورات بعدی قابل استفاده هستند.

مرحله چهارم : پیاده سازی MEM Stage



پردازنده در **مرحله Memory** با سیستم حافظه تعامل دارد. در این مرحله، پردازنده برای دستورات load داده‌ها را از حافظه خوانده و آن‌ها را در یک رجیستر ذخیره می‌کند. برای دستورات store، پردازنده داده‌ها را در حافظه می‌نویسد. این مرحله با استفاده از ماژول حافظه آماده (Distributed Memory Generator) که در vivado بود انجام شد.

مرحله پنجم : پیاده سازی Hazard Unit

در این مرحله، **ماژول تشخیص و مدیریت مخاطره‌ها** (Hazard Unit) به پردازنده اضافه می‌شود. این ماژول وظیفه تشخیص و رفع مخاطره‌های داده‌ای، به ویژه مخاطره‌های نوع RAW (Read After Write) را بر عهده دارد. در پردازنده‌ها به طور کلی سه نوع مخاطره وجود دارد که باید مدیریت شوند:

الف) **مخاطره ساختاری**: این نوع مخاطره ناشی از ساختار خط لوله پردازنده است و به همین دلیل به آن مخاطره ساختاری گفته می‌شود. در این پردازنده، این مخاطره بین مراحل WB و ID رخ می‌دهد، زیرا عملیات خواندن و نوشتن از رجیسترهای عمومی به صورت همزمان انجام می‌شود. برای رفع این مشکل، عملیات نوشتن در رجیسترهای عمومی به لبه پایین رونده کلاک منتقل شده است، که این مخاطره را به طور کامل رفع می‌کند.

ب) **مخاطره کنترلی**: این مخاطره ناشی از دستورات پرش است. هنگامی که دستور پرش وارد خط لوله می‌شود، به دلیل تاخیر در تشخیص و محاسبه آدرس پرش، دو دستور اشتباه به خط لوله وارد می‌شوند. برای رفع این مشکل، سیگنالهای کنترلی Flush به پردازنده اضافه شده‌اند که دستورات اشتباه را از خط لوله حذف می‌کنند.

ج) **مخاطره داده ای**: این نوع مخاطره به تعاملات داده ها مربوط میشود و به سه حالت تقسیم می شود:

1. خواندن پس از نوشتن: (Read After Write)
2. نوشتن پس از خواندن: (Write After Read)
3. نوشتن پس از نوشتن: (Write After Write)

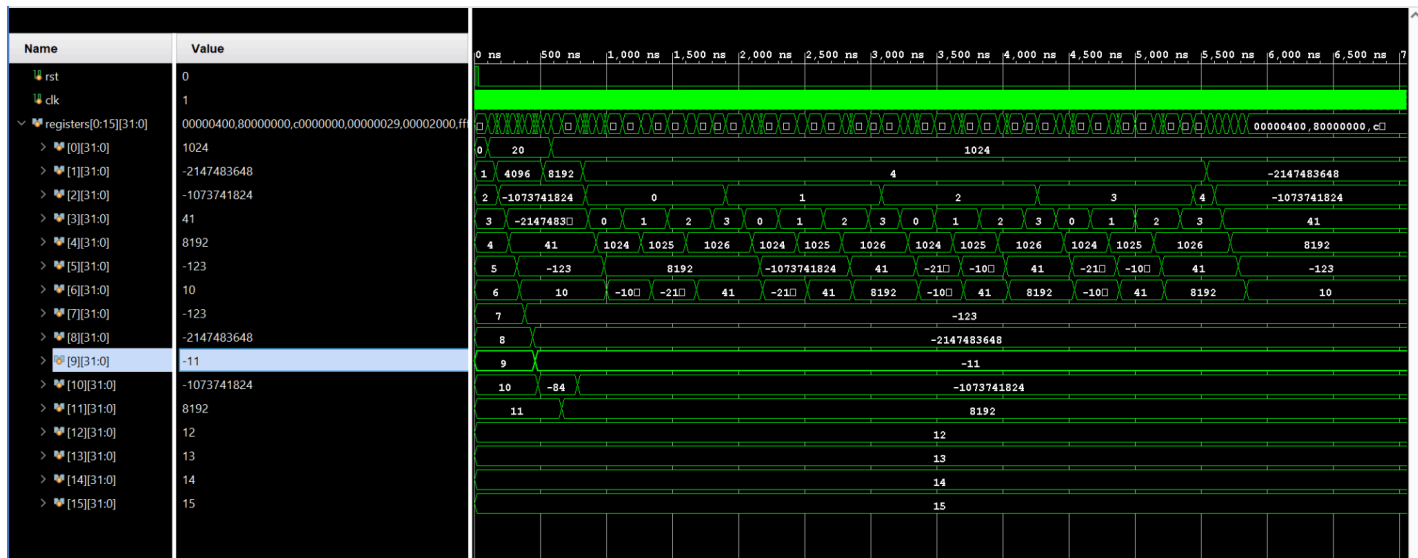
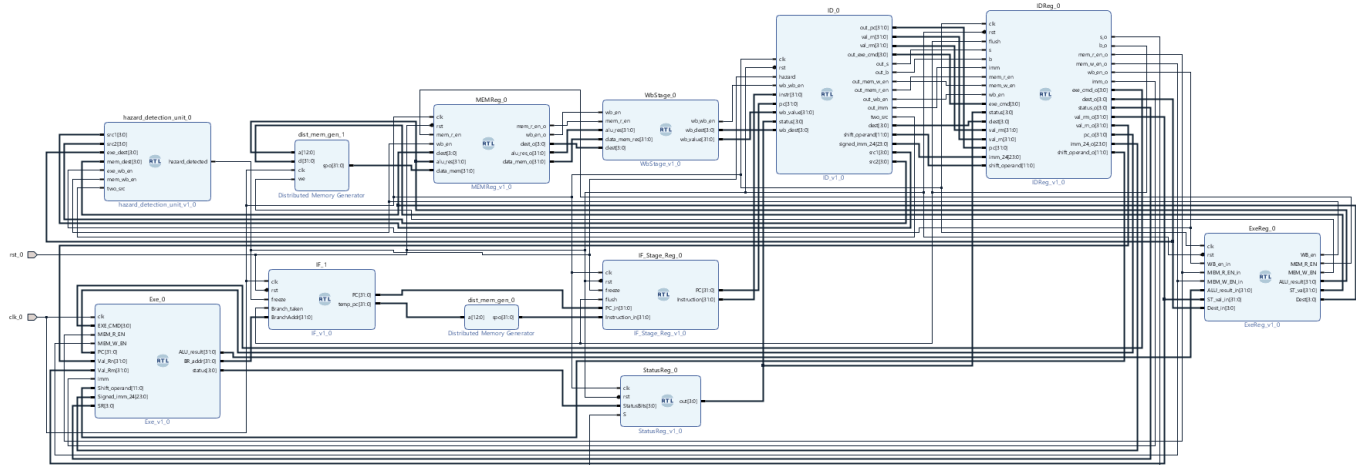
مخاطره RAW زمانی رخ می دهد که یک دستور نیاز به خواندن از رجیستری دارد که هنوز توسط دستور قبلی نوشته نشده است. برای رفع این مشکل، Hazard Unit آدرس رجیسترها (src1 , src2) در مرحله ID را با مقصدهای دستورات در مراحل EXE و MEM مقایسه می کند. اگر یکی از منابع با مقصد دستورات در این مراحل برابر باشد و سیگنال EN_WB فعال باشد، مخاطره تشخیص داده می شود و سیگنال hazard فعال می شود.

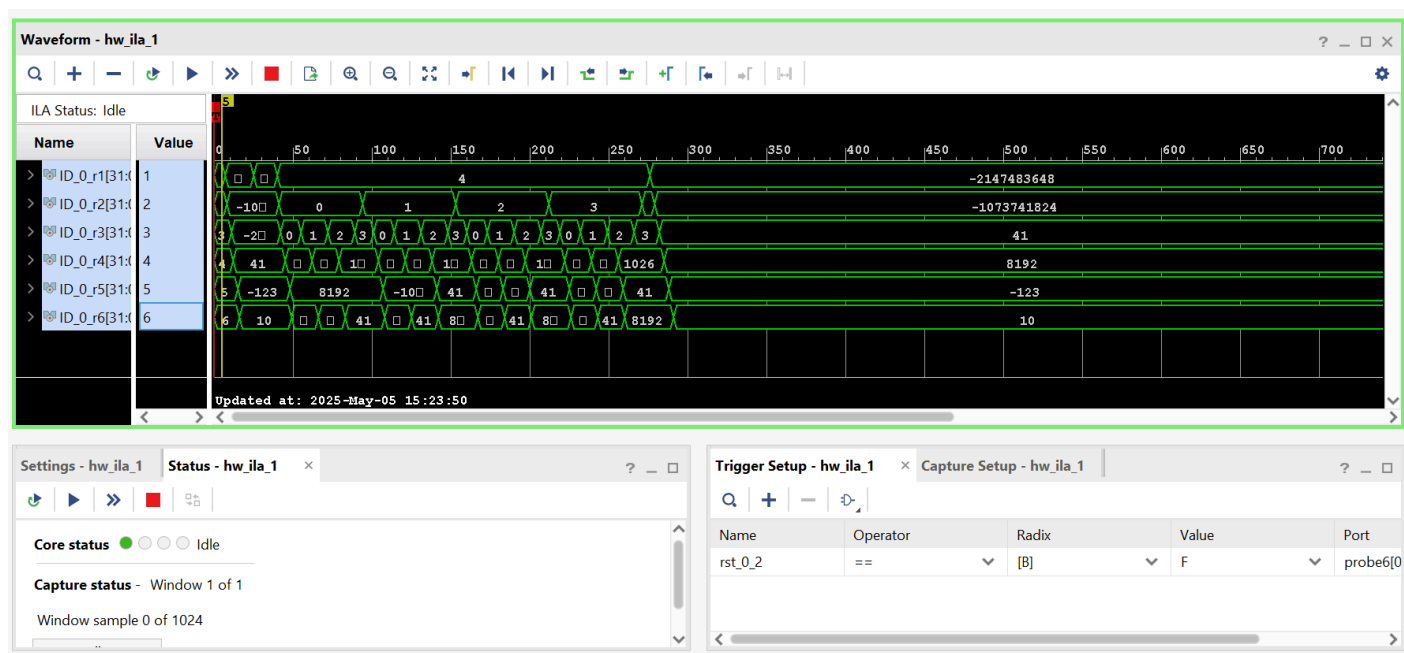
هنگامی که مخاطره تشخیص داده می شود، دستورات درون IF و رجیسترهای پس از آن متوقف می شوند و یک حباب در خط لوله ایجاد می شود. این کار با صفر کردن سیگنال های کنترلی خروجی از واحد کنترل انجام می شود. برای پیاده سازی این مرحله، ورودی های مورد نیاز Hazard Unit از مراحل مختلف خط لوله تأمین می شوند و خروجی آن به سیگنال Freeze در رجیستر PC و رجیسترهای پس از IF متصل می شود. این مرحله آخرین بخش از پیاده سازی پردازنده ARM است و قسمت های بعدی شامل ویژگی های اضافی است که به ARM اضافه می شوند.

نتایج نهایی پیاده‌سازی ARM

پس از پایان مرحله‌ی 5، برنامه محک را با پردازنده اجرا کردیم و مشاهده کردیم که اعداد به درستی مرتب شدند و برنامه بدون مشکل به خروجی مدنظر رسید. خروجی simulation و On board آن را می‌توانید در زیر مشاهده کنید.

:simulation





مرحله ششم : پیاده سازی Forwarding Unit

ماژول [Forwarding Unit](#) برای مدیریت تداخل‌های داده‌ای (data hazards) در پایپ‌لاین پردازنده طراحی شده است. پس از تکمیل پنج مرحله پردازنده، همان‌طور که پیشتر اشاره شد، ممکن است یک دستور به محتوای رجیسترهایی نیاز داشته باشد که هنوز اطلاعات آنها به روز رسانی نشده است. این مشکل از آنجا ناشی می‌شود که داده‌ها پس از پایان مرحله Write Back به رجیستر فایل منتقل می‌شوند. با این حال، می‌دانیم که داده مورد نظر پس از اتمام مرحله EXE آماده است و تنها در دو مرحله دیگر به رجیستر مربوطه نوشته می‌شود. در حالت عادی، برای دستیابی به این داده باید پایپ‌لاین را برای چند سیکل متوقف کنیم، که این امر به شدت کارایی سیستم را کاهش می‌دهد. برای حل این مشکل، به جای خواندن محتوا از رجیستر، داده‌ای که قرار است در آن نوشته شود را مستقیماً پس از مرحله EXE دریافت کرده و forward می‌کنیم. برای انجام این عملیات، ماژولی به نام Forwarding Unit طراحی شده است که وظیفه انتقال مستقیم داده از مرحله EXE را بر عهده دارد و به این ترتیب، از توقف پایپ‌لاین جلوگیری می‌کند و کارایی سیستم را بهبود می‌بخشد.

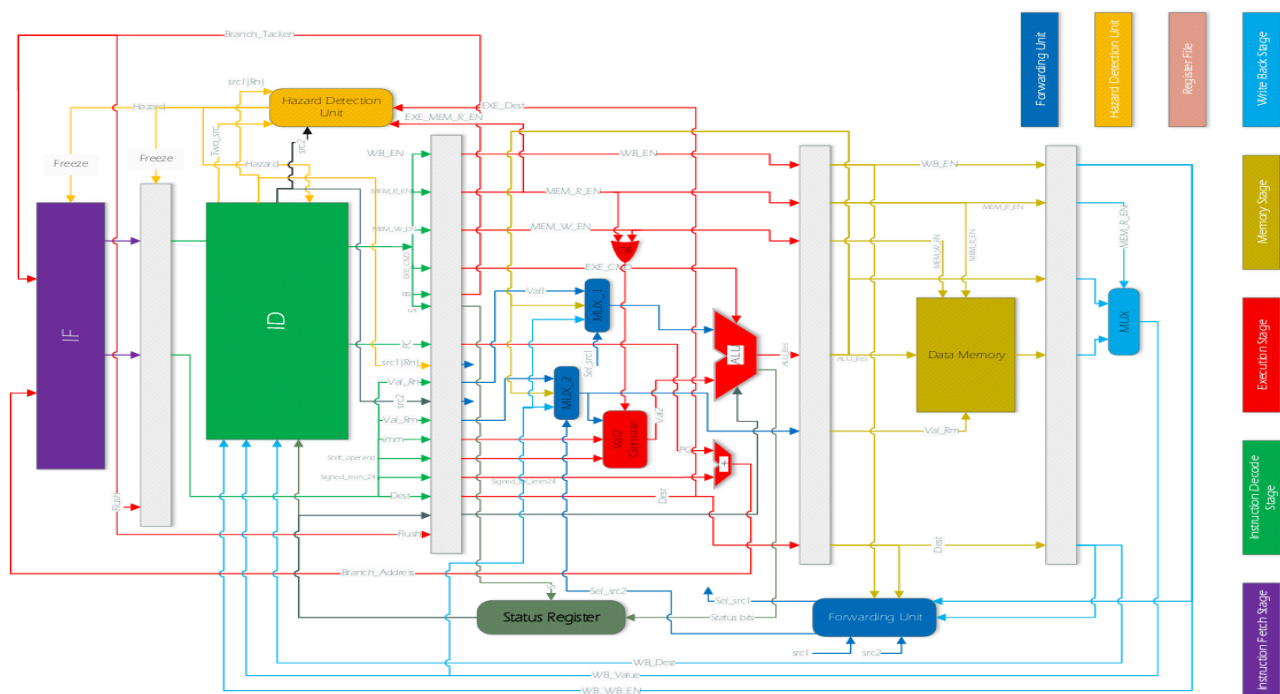
این ماژول وظیفه مدیریت forwarding را بر عهده دارد و سیگنال ورودی forward_en تعیین می‌کند که آیا ماژول فعال است یا خیر. در صورتی که این سیگنال غیرفعال باشد، پردازنده مانند حالتی عمل می‌کند که forwarding وجود ندارد، و تمامی تداخل‌ها (hazards) توسط ماژول Hazard Unit مدیریت می‌شوند.

این ماژول دو سیگنال خروجی به نام‌های sel_src1 و sel_src2 تولید می‌کند که به mux های مرحله EXE متصل هستند. این mux ها بین سه منبع داده تصمیم‌گیری می‌کنند: مقدار رجیستر از مرحله ID، مقدار به‌روز

شده توسط ALU از مرحله MEM ، و مقدار نهایی از مرحله WB. اگر Forwarding Unit فعال باشد، داده‌های صحیح از مراحل بعدی پایپ‌لاین انتخاب می‌شوند. در غیر این صورت، مقدار رجیستر از مرحله ID استفاده می‌شود.

در صورت عدم وجود Forwarding Unit، تمامی تداخل‌های داده‌ای توسط Hazard Unit تشخیص داده شده و با stall کردن پایپ‌لاین مدیریت می‌شدند. با این حال، Forwarding Unit این امکان را فراهم می‌کند که به جای استال کردن، داده‌ها مستقیماً از مراحل بعدی به مراحل قبلی منتقل شوند. تنها استثنا دستور LDR است که به دلیل عدم پایداری داده‌های خوانده شده از حافظه، نیاز به استال کردن پایپ‌لاین دارد.

این مکانیزم باعث بهبود عملکرد پردازنده و کاهش تاخیرهای ناشی از تداخل‌های داده‌ای می‌شود.

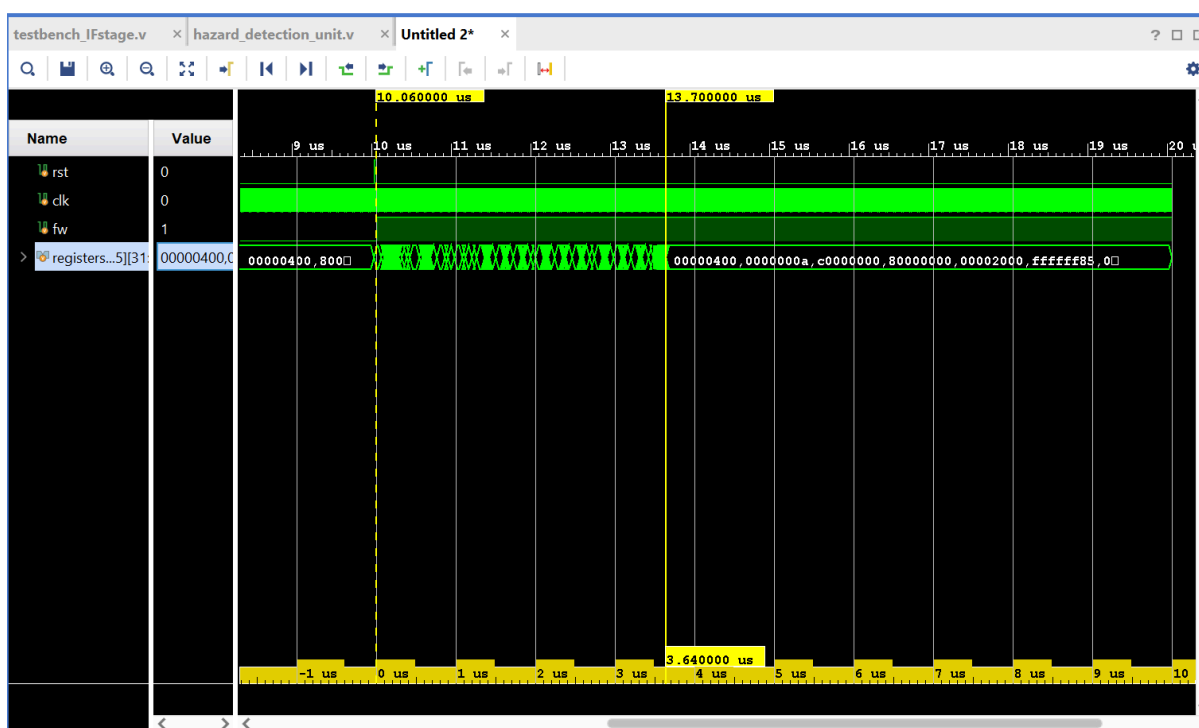


حال برنامه را با دو حالت فعال بودن و غیرفعال بودن forwarding تست میکنیم تا میزان افزایش کارایی را بدست آوریم.

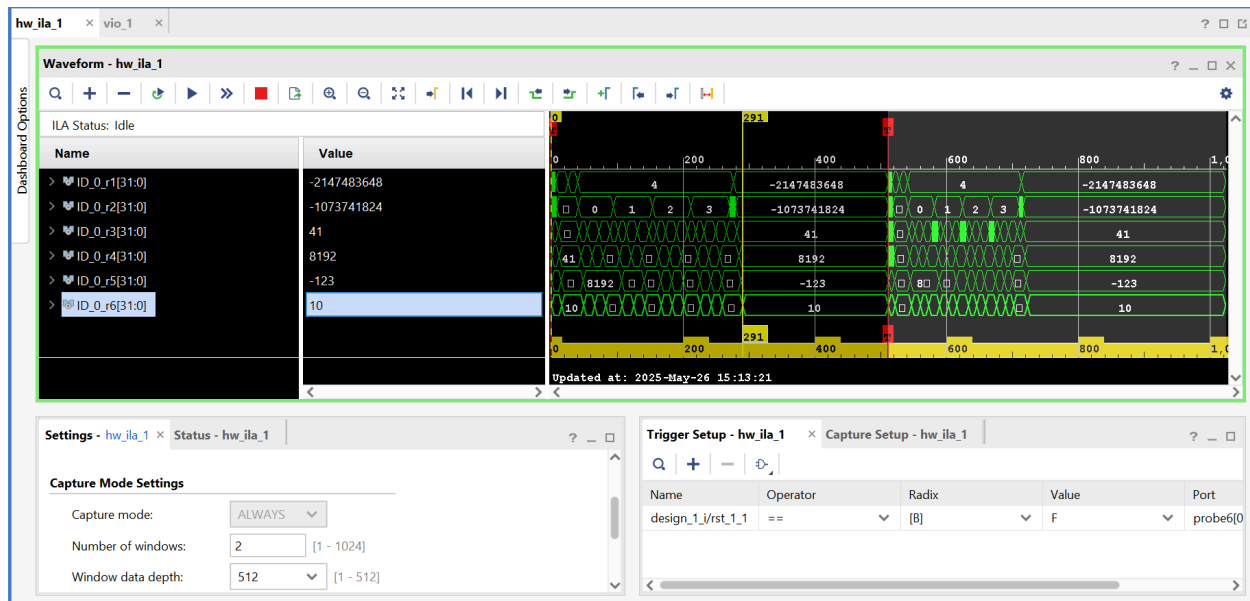
خروجی با استفاده از Forwarding:



خروجی بدون استفاده از Forwarding:



هر دو خروجی با هم روی board:



همان‌طور که در دو تصویر اول مشاهده می‌شود مشاهده می‌شود، زمان اجرای برنامه در حالت غیر فعال بودن Forwarding برابر 10.06 میکرو ثانیه و در حالت فعال‌سازی برابر 5.86 میکرو ثانیه شده است. با محاسبه بهبود عملکرد، به نتیجه زیر می‌رسیم:

$$\frac{10.06-5.86}{10.06} \times 100 \cong 41.74\% \text{ improvement}$$

این بهبود 41.74 درصدی نشان‌دهنده تاثیر مثبت استفاده از Forwarding در کاهش زمان اجرای برنامه و افزایش کارایی پردازنده است.

بخش امتیازی:

برای افزایش کارایی پردازنده به جای استفاده از Forwarding Unit، میتوان از روشهای زیر بهره برد:

1. پیاده سازی سیستم پیش بینی (Branch Prediction):

با اضافه کردن یک ماژول پیش بینی Branch در پردازنده، میتوان از ورود دستورات اشتباه به خط لوله جلوگیری کرد. این سیستم پیش‌بینی میکند که آیا Branch رخ خواهد داد یا نه، و آدرس هدف را به صورت زود هنگام محاسبه میکند. استفاده از این الگوریتم ها می تواند تعداد استال ها را کاهش داده و عملکرد کلی پردازنده را بهبود بخشد.

2. اجرای نامنظم دستورات (Out of Order Execution):

در این روش، پردازنده دستورات را به ترتیبی که وابستگی‌های داده‌ای و کنترلی اجازه می‌دهند اجرا می‌کند، نه لزوماً به ترتیبی که در برنامه نوشته شده‌اند. این روش با استفاده از Reorder Buffer و تحلیل وابستگی دستورات، امکان اجرای دستورات غیرمرتبط به صورت جلوتر را فراهم می‌کند و عملکرد کلی پردازنده را بهبود می‌بخشد. این کار باعث می‌شود که دستورات مستقل از یکدیگر به صورت موازی اجرا شوند و از اسنال جلوگیری شود.

3. استفاده از دستورات SIMD (Single Instruction, Multiple Data):

این دستورات به پردازنده اجازه می‌دهند تا یک عملیات را روی چندین داده به طور همزمان انجام دهد. این روش به ویژه برای برنامه‌هایی که نیاز به پردازش موازی داده‌ها دارند، مانند پردازش تصویر یا محاسبات ماتریسی، بسیار مفید است.

4. استفاده از Cache:

با کاهش زمان دسترسی به حافظه اصلی، سرعت اجرای برنامه‌ها را افزایش می‌دهد.